security/05/app/controllers/tasks_controller.rb
```
def task_params
  params.require(:task).permit(:project_id, :title, :size)
end
```

We manage the strong-parameters issue in other ways. The ProjectsController#create method explicitly lists the items that are being passed to CreatesProject:

security/05/app/controllers/projects_controller.rb
```
def create
  @workflow = CreatesProject.new(
    name: params[:project][:name],
    task_string: params[:project][:tasks],
    users: [current_user])
  @workflow.create
  if @workflow.success?
    redirect_to projects_path
  else
    @project = @workflow.project
    render :new
  end
end
```

As with other features the framework provides, the important part from the testing framework is the behavior—preventing the user from setting a particular attribute—rather than the implementation.

Prescription 34 Test for mass assignment any time you have an attribute that needs to be secure and a controller method that touches that class based on user input.

Normally in code I do one of two things: use strong parameters or pass an explicit list of parameters to a workflow object.

Testing strong parameters often has the same problem as the previous security issue: integration tests don't work because the potential problem is not accessible via the UI, so Capybara can't be used to trigger a test failure. Request specs can again be useful here, since you can send arbitrary hashes as the parameters.

Sometimes you'll pass the params object directly to the workflow and need to do the strong-parameter check there, in which case you have a problem when testing because the params object isn't a hash. You need to create the underlying Rails object directly:

```
CreatesProject.new(ActionController::Parameters.new(name: "Project"))
```